

Guia de Inicialização do Ambiente de Desenvolvimento

Smart Weather Platform — WSL2 + Docker + Laravel

Este documento é o seu passo a passo de referência para começar a trabalhar no projeto todos os dias, a partir da migração feita em julho/2026 para o WSL2 (Ubuntu).

1. Entendendo o ambiente (leia uma vez, para não se confundir depois)

Depois da migração, existem **duas cópias físicas** do projeto no seu computador:

	Local antigo (Windows)	Local atual (WSL2/Linux)
Caminho	C:\Users\renat\smart-weather-platform	/home/renato/smart-weather-platform
Status	Parado, não usar mais	Ativo — é aqui que você trabalha
Terminal	Git Bash	Terminal Ubuntu
Velocidade	Lenta (3-7s por página)	Rápida (~0.07s por página)

Regra de ouro: a partir de agora, todo comando, toda edição de código, todo `git`, `docker`, `npm`, `composer` acontece dentro do terminal **Ubuntu**, na pasta `/home/renato/smart-weather-platform`. A pasta do Windows pode ficar esquecida como backup histórico — não abra ela, não edite nada lá.

Sobre inicialização automática

- WSL2 (Ubuntu):** não precisa "ligar" manualmente. Ele inicia sozinho no instante em que você abre um terminal Ubuntu ou digita `ws1` em qualquer terminal do Windows.
- Docker Desktop:** geralmente abre sozinho ao ligar o Windows (configurável em Settings → General). Confirme sempre que o ícone da baleia 🐳 está ativo na bandeja do sistema (canto inferior direito da tela) antes de começar a trabalhar.
- Containers do projeto (MySQL, Redis, etc.):** têm a política `restart: unless-stopped`, então costumam voltar sozinhos se o Docker Desktop reiniciar — mas **sempre confirme** com `docker compose ps` antes de assumir que estão de pé.

2. Passo a passo diário (do zero até codar)

Passo 1 — Abrir o terminal Ubuntu

Use qualquer uma dessas formas:

- Menu Iniciar do Windows → digitar "**Ubuntu**" → abrir o app
- Ou abrir o PowerShell/CMD e digitar:

```
wsl -d Ubuntu
```

Você deve ver um prompt parecido com:

```
renato@DESKTOP-RENATO:~$
```

Passo 2 — Ir até a pasta do projeto

```
bash  
cd ~/smart-weather-platform
```

(o `~` já representa `/home/renato`, então isso equivale a `cd /home/renato/smart-weather-platform`)

Passo 3 — Confirmar/subir os containers Docker

Primeiro, veja o que já está rodando:

```
bash  
docker compose ps
```

- Se todos os serviços aparecerem como "**Up**" (e os que têm healthcheck como "**healthy**") → pode pular para o Passo 4.
- Se aparecer vazio, parcial, ou algum container "Exited" → suba tudo:

```
bash  
docker compose up -d
```

Aguarde uns 10-15 segundos (o MySQL demora um pouco para ficar "healthy" na primeira subida do dia) e confirme de novo com `docker compose ps`.

Passo 4 — Garantir que o Node.js está ativo (se for mexer no frontend)

O Node foi instalado via `nvm` (gerenciador de versões). Normalmente ele carrega sozinho,

mas se algum comando `npm`/`vite` der erro de "command not found", rode:

```
bash  
  
nvm use 22
```

Para verificar qual Node está ativo a qualquer momento:

```
bash  
  
node -v  
npm -v  
which node
```

O caminho do `which node` deve começar com `/home/renato/.nvm/...` — se aparecer `/mnt/c/...`, é o Node errado (do Windows).

Passo 5 — Abrir o VS Code corretamente

Nunca abra o VS Code clicando no ícone e navegando manualmente até a pasta do Windows. Sempre abra a partir do terminal Ubuntu, de dentro da pasta do projeto:

```
bash  
  
code .
```

Confirme que abriu certo: no **canto inferior esquerdo** da janela do VS Code deve aparecer uma tag verde escrito "**WSL: Ubuntu**". Se não aparecer isso, você está editando a pasta errada — feche e repita o comando `code .` de dentro do terminal Ubuntu.

O terminal integrado do VS Code (atalho `Ctrl+``) já abre automaticamente como terminal Ubuntu nessa configuração — você pode rodar todos os comandos artisan/npm/docker direto por ali também.

Passo 6 — Acessar o sistema no navegador

```
http://127.0.0.1:8000
```

Nota: prefira `127.0.0.1` em vez de `localhost` — identificamos que `localhost` pode tentar resolver por IPv6 e falhar em alguns casos neste ambiente. `127.0.0.1` sempre força IPv4 e funciona de forma consistente.

Outras URLs úteis:

- Dashboard: `http://127.0.0.1:8000/dashboard`
- Mailpit (emails capturados): `http://127.0.0.1:8000:8025` → na verdade é `http://127.0.0.1:8025`

3. Checklist rápido (resumo dos passos 1-6)

```
bash

# 1. Abrir terminal Ubuntu (menu Iniciar → Ubuntu)

# 2. Ir para o projeto
cd ~/smart-weather-platform

# 3. Conferir/subir containers
docker compose ps
docker compose up -d    # só se necessário

# 4. Garantir Node certo (se for mexer em frontend)
nvm use 22

# 5. Abrir VS Code
code .

# 6. Acessar no navegador
# http://127.0.0.1:8000
```

4. Comandos do dia a dia (referência rápida)

Backend (Laravel)

```
bash

docker compose exec app php artisan migrate      # rodar migrations
docker compose exec app php artisan tinker      # console interativo
docker compose exec app php artisan test        # rodar testes automati
docker compose exec app php artisan route:list   # listar rotas
docker compose exec app php artisan config:clear # limpar cache de con
docker compose exec app php artisan view:clear   # limpar cache de vi
```

Frontend (React/Vite)

```
bash

cd backend
npm run dev      # servidor de desenvolvimento com hot-reload
npm run build    # build de produção (necessário após qualquer mudança em .j
```

Git

```
bash

git status
git add .
git commit -m "mensagem descritiva"
git push origin develop
```

Docker (manutenção geral)

```
bash

docker compose ps                # ver status dos containers
docker compose logs app --tail=50 # ver logs do backend
docker compose logs nginx --tail=50 # ver logs do servidor web
docker compose restart nginx      # reiniciar só o nginx
docker compose build app           # reconstruir a imagem do backend (após
docker compose down               # desligar tudo (fim do dia, opcional)
```

5. Problemas conhecidos e soluções rápidas

"vite: não é reconhecido" ou erros de caminho UNC no `npm run build`

Você está usando o Node do Windows sem querer. Rode:

```
bash

nvm use 22
which npm # deve mostrar /home/renato/.nvm/...
```

Site lento de novo (voltou a demorar segundos para carregar)

Confirme que você está de fato dentro do WSL2 (`pwd` deve mostrar `/home/renato/...`, não `/mnt/c/...`). Se voltar a acontecer mesmo assim, reinicie o WSL2:

```
powershell

# No PowerShell do Windows, como Administrador:
wsl --shutdown
```

Depois reabra o Docker Desktop e o terminal Ubuntu.

Erro 504 Gateway Time-out ou ERR_EMPTY_RESPONSE logo após `docker compose build app`

O Nginx ficou com o IP antigo do container `app` em cache. Reinicie o Nginx:

```
bash
docker compose restart nginx
```

Erro de permissão ao carregar páginas (`tempnam(): file created...` ou views não compilam)

As pastas de storage ficaram com o dono errado após um `git clone`/`git pull`. Corrija com:

```
bash
docker compose exec app chown -R www-data:www-data storage bootstrap/cache
docker compose exec app chmod -R 775 storage bootstrap/cache
docker compose exec app php artisan view:clear
```

`git push` pedindo senha e recusando

O GitHub não aceita mais senha normal via HTTPS. Use um **Personal Access Token** (gerado em <https://github.com/settings/tokens>, tipo "classic", com escopo `repo` marcado) no lugar da senha. Depois do primeiro uso bem-sucedido, o comando abaixo evita ter que repetir isso:

```
bash
git config --global credential.helper store
```

6. Fim do dia (opcional)

Você **não é obrigado** a desligar os containers ao final do dia — pode deixar tudo rodando e só fechar o VS Code e os terminais. Mas se quiser liberar recursos da máquina:

```
bash
docker compose down
```

Na próxima vez, basta repetir o passo a passo da Seção 2 (`docker compose up -d` sobe tudo de novo).

Documento gerado em 02/07/2026, referente à migração do ambiente de desenvolvimento de Windows nativo para WSL2 (Ubuntu), realizada para resolver lentidão severa causada pelo bind mount entre NTFS e o filesystem Linux dos containers Docker.